Documentación del servidor XAMPP para el proyecto NEJO

Autores: Johan Romero Duarte, Néider Forero Molina

Profesor: Jhonatan Paolo Tovar Soto

Programa: Tecnología en Automatización Industrial

Institución: Universidad San Buenaventura Sede Bogotá D.C.

Semestre y Año: Semestre V, 2026

Servidor local de recepción y visualización de datos de sensores (IIoT)

1. Resumen

Este documento describe el servidor local construido con XAMPP (Apache + PHP + MySQL) para el proyecto NEJO. Su propósito es recibir las lecturas de temperatura y humedad enviadas por el microcontrolador ESP32, almacenarlas en una base de datos y mostrarlas en un panel web (dashboard). Con esto el proyecto pasa de leer los sensores por monitor serie o Bluetooth a contar con telemetría centralizada e histórica, que es uno de los ejes planteados en el objetivo general del proyecto (consolidación del hardware de control y la telemetría de datos).

2. Objetivo

Objetivo general:

Implementar un servidor local con XAMPP que reciba, almacene y visualice en tiempo casi real los datos de los sensores del invernadero NEJO.

Objetivos específicos:

- Configurar un entorno de servidor local con Apache, PHP y MySQL mediante XAMPP.
- Crear una base de datos (iiot_sensores) que guarde el historial de lecturas con su fecha y hora.
- Desarrollar un script receptor (recibir.php) que reciba datos del ESP32 por HTTP (método POST) y los inserte en la base de datos.
- Desarrollar un dashboard (dashboard.php) que muestre las últimas lecturas registradas.

3. Contexto dentro del proyecto NEJO

NEJO busca mantener condiciones óptimas de temperatura y humedad en un invernadero universitario mediante automatización con ESP32. En las etapas anteriores se trabajó con:

- Sensor SHT3X (I2C, dirección 0x44) para temperatura y humedad.
- Sensor PT100 con puente de Wheatstone, leído por el ADC del ESP32 (pin 34, resolución de 12 bits) y convertido a °C con una ecuación lineal.
- Comunicación Bluetooth (nombre del dispositivo ESP32CJN) para enviar las lecturas a un dispositivo cercano.

El servidor XAMPP añade la comunicación por Wi-Fi y el almacenamiento persistente, de modo que los datos queden guardados y consultables desde cualquier navegador de la red local.

4. Arquitectura del sistema

El flujo de datos es el siguiente:

1. Sensores (SHT3X / PT100) miden temperatura y humedad.
2. El ESP32 lee los sensores y envía una petición HTTP POST con los campos sensor y valor a la dirección del servidor (por ejemplo http://IP_DEL_PC/recibir.php).
3. Apache + PHP ejecuta recibir.php, que valida los datos.
4. MySQL guarda la lectura en la tabla lecturas de la base iiot_sensores.
5. El usuario abre dashboard.php en el navegador y ve las últimas 20 lecturas.

Componente	Función
ESP32	Lee sensores y envía los datos por Wi-Fi
Apache (XAMPP)	Servidor web que atiende las peticiones HTTP
PHP	Lógica del servidor: recibir, validar y consultar
MySQL / MariaDB	Almacenamiento de las lecturas
Navegador	Visualización del dashboard

5. Instalación y configuración de XAMPP

1. Descargar e instalar XAMPP (Windows) desde el sitio oficial de Apache Friends.
2. Abrir el Panel de control de XAMPP e iniciar los módulos Apache y MySQL.
3. Copiar los archivos recibir.php y dashboard.php en la carpeta pública del servidor, normalmente C:\xampp\htdocs\ (o en una subcarpeta, por ejemplo htdocs\nejo\).
4. Abrir phpMyAdmin (http://localhost/phpmyadmin) y crear la base de datos y la tabla (sección 6).
5. Probar en el navegador: http://localhost/dashboard.php.
6. Para que el ESP32 se conecte, el computador y el ESP32 deben estar en la misma red Wi-Fi. Se consulta la IP del computador con ipconfig y se usa esa IP en el código del ESP32. Puede ser necesario permitir Apache (puerto 80) en el firewall de Windows.

6. Base de datos

Nombre de la base de datos: iiot_sensores **Tabla:** lecturas

Según las consultas de los scripts, la tabla maneja los campos id, sensor, valor y fecha. A continuación se propone una estructura compatible (ajustarla si la tabla real creada difiere):

```
CREATE DATABASE IF NOT EXISTS iiot_sensores;
USE iiot_sensores;

CREATE TABLE IF NOT EXISTS lecturas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  sensor VARCHAR(50) NOT NULL,
  valor VARCHAR(20) NOT NULL,
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Campo	Descripción
id	Identificador autoincremental de la lectura
sensor	Nombre del sensor (p. ej. temperatura, humedad)
valor	Valor medido
fecha	Fecha y hora de registro, asignada automáticamente por MySQL

7. Descripción de los archivos

7.1 recibir.php (receptor de datos)

Recibe los datos enviados por el ESP32 y los guarda en la base de datos.

- Se conecta a MySQL con mysqli (servidor localhost, base iiot_sensores) y verifica que la conexión sea exitosa.
- Lee las variables sensor y valor enviadas por POST.

- Si ambos tienen contenido, usa una consulta preparada (prepare + bind_param) para insertar la lectura, lo que protege contra inyección SQL.
- Responde con un mensaje de texto: "Datos guardados correctamente", un mensaje de error de la base de datos, o "Faltan datos del sensor o valor".

Ejemplo de petición (desde terminal, para probar sin el ESP32):

```
curl -X POST -d "sensor=temperatura" -d "valor=23.5" http://localhost/recibir.php
```

7.2 dashboard.php (panel de visualización)

Muestra en una tabla HTML las últimas 20 lecturas (ORDER BY id DESC LIMIT 20) con las columnas Sensor, Valor y Fecha. Se actualiza al recargar la página.

7.3 launch.json (configuración de VS Code)

Archivo generado por la extensión *Microsoft Edge DevTools* de Visual Studio Code. Define cómo lanzar Edge en modo normal o *headless* con depuración remota (puerto 9222). Es solo configuración del entorno de desarrollo y no interviene en el funcionamiento del servidor.

8. Prueba de funcionamiento

1. Iniciar Apache y MySQL en XAMPP.
2. Enviar una lectura de prueba con curl o desde el ESP32.
3. Verificar en phpMyAdmin que se creó una fila nueva en lecturas.

4. Abrir http://localhost/dashboard.php y confirmar que aparece la lectura.

Se recomienda agregar aquí capturas de pantalla del panel de XAMPP, de phpMyAdmin y del dashboard como evidencia.

9. Conexión con el ESP32

Los archivos entregados no incluyen el código del ESP32 que envía los datos al servidor. Su estructura esperada es: conectarse al Wi-Fi con la librería WiFi.h, y enviar un POST con HTTPClient a http://IP_DEL_PC/recibir.php con el cuerpo sensor=temperatura&valor=23.5 y el encabezado application/x-www-form-urlencoded. Conviene incluir ese código en el informe final.

10. Limitaciones y mejoras recomendadas

- Seguridad de acceso a la base de datos: se usa el usuario root sin contraseña, válido solo para pruebas locales. En un entorno real debe crearse un usuario con permisos limitados y contraseña.
- Validación con empty(): en PHP, empty("0") es verdadero, por lo que una lectura con valor exactamente 0 sería rechazada. Se sugiere usar $valor !== ''.
- Salida en el dashboard: los datos se imprimen sin escapar; se recomienda usar htmlspecialchars() para evitar inyección de código en la página.

- Tipo del campo valor: guardarlo como número (DECIMAL) facilitaría promedios, gráficas y alarmas.
- Actualización automática: agregar <meta http-equiv="refresh" content="5"> al dashboard o usar JavaScript para refrescar los datos.
- Visualización: incorporar gráficas (por ejemplo con Chart.js) de temperatura y humedad en el tiempo.
- Control de actuadores: extender el sistema para que el servidor registre o envíe órdenes al ESP32 (ventilador, bomba) según umbrales.

11. Conclusiones

El servidor XAMPP permite que el proyecto NEJO cuente con una cadena completa de adquisición, transmisión, almacenamiento y visualización de datos, aplicando conceptos de IIoT con herramientas de bajo costo. Sienta la base para las siguientes etapas: gráficas históricas, alarmas y control remoto del microclima del invernadero.